#12

Controlling the PC

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

Class announcements

- **If you had a negative grade in the project**
  - **You should talk to me**

Assemblers and Compilers

How the machine-code sausage is made

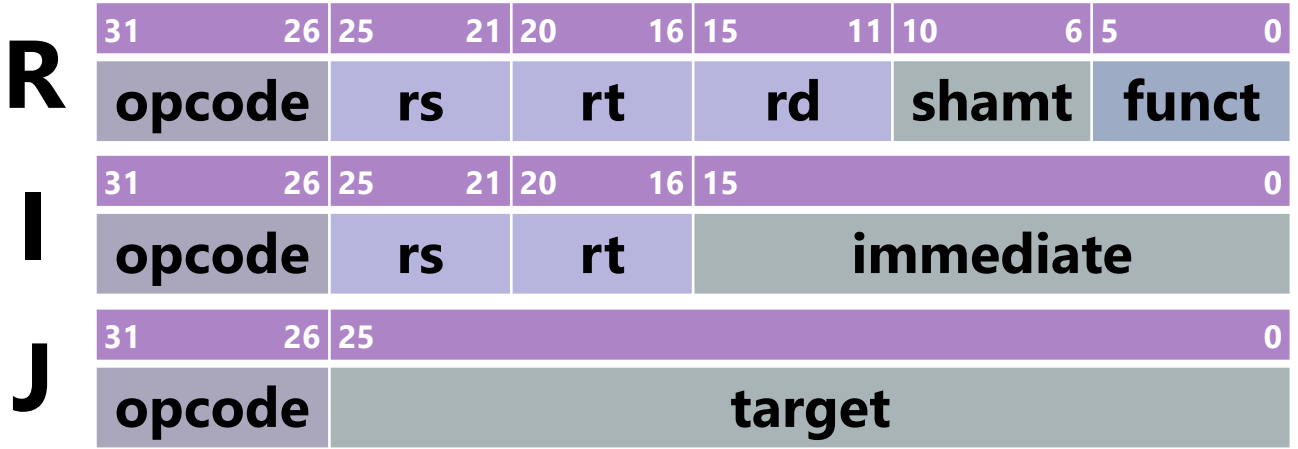
What is machine code?

Something denser

- text is human-oriented and informationally… sparse.
- instead, we encode each instruction as a **bitfield.**
  - this encoding is specified by the ISA.

MIPS has three **instruction formats.**

the **opcode** (and **funct**) field identifies *which instruction* it is.




```
add rd,rs,rt
sll rd,rs,shamt

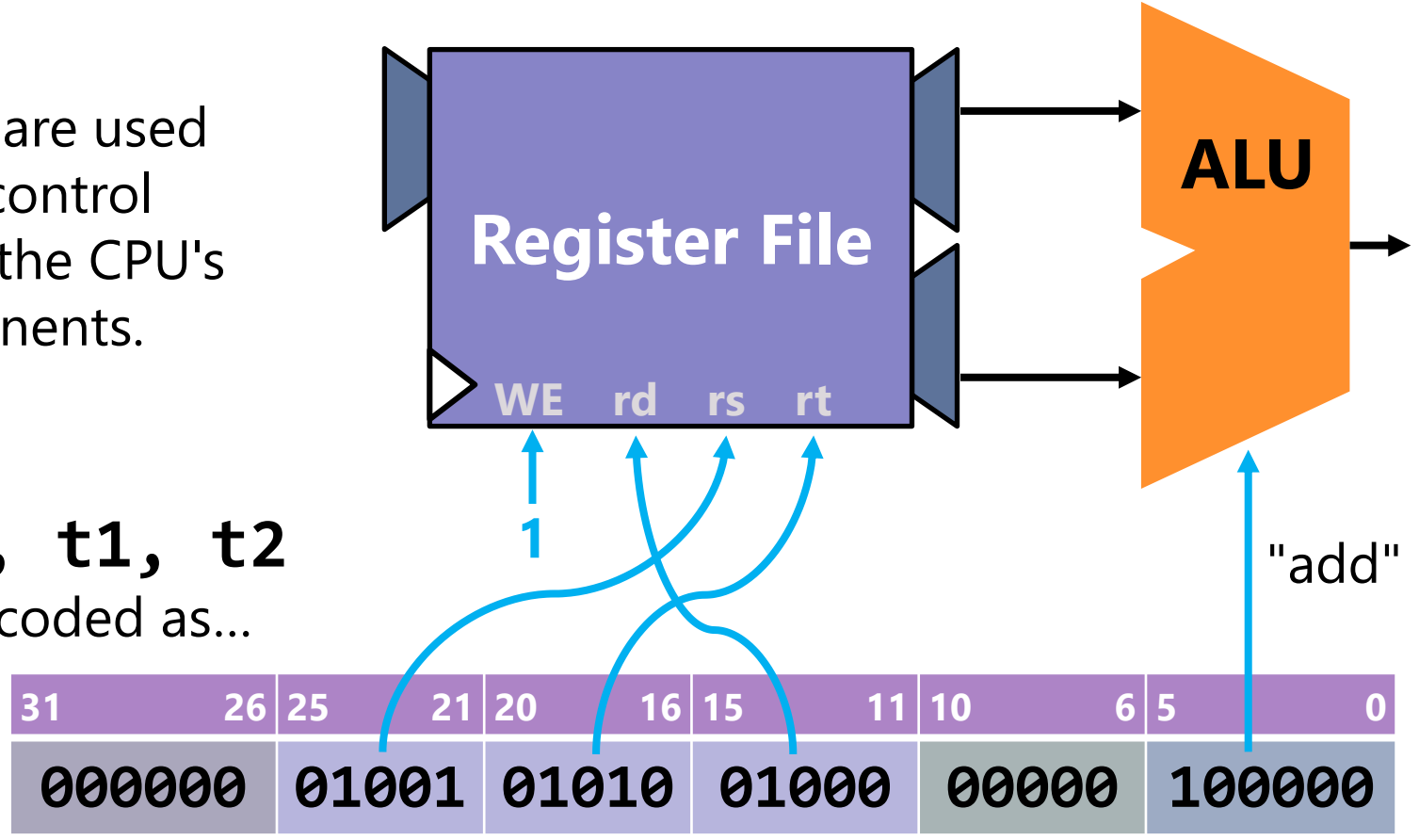
beq rs,rt,offset

jal target
```

How does it... do the thing?

- well this is mostly next lecture, but...

the fields are used as the control signals to the CPU's components.

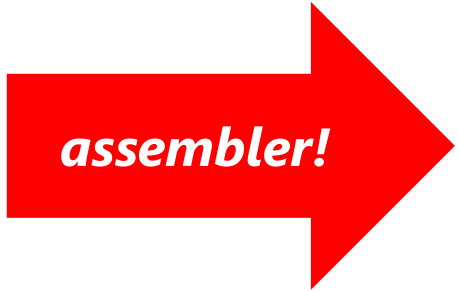


`add t0, t1, t2`
gets encoded as...

How It's Made

So we know the assembler

```
    li   s0, 0
top:
    move a0, s0
    jal  print_int
    addi s0, s0, 1
    blt  s0, 5, top
    li   v0, 10
    syscall
print_int:
    li v0, 1
    syscall
    jr ra
```



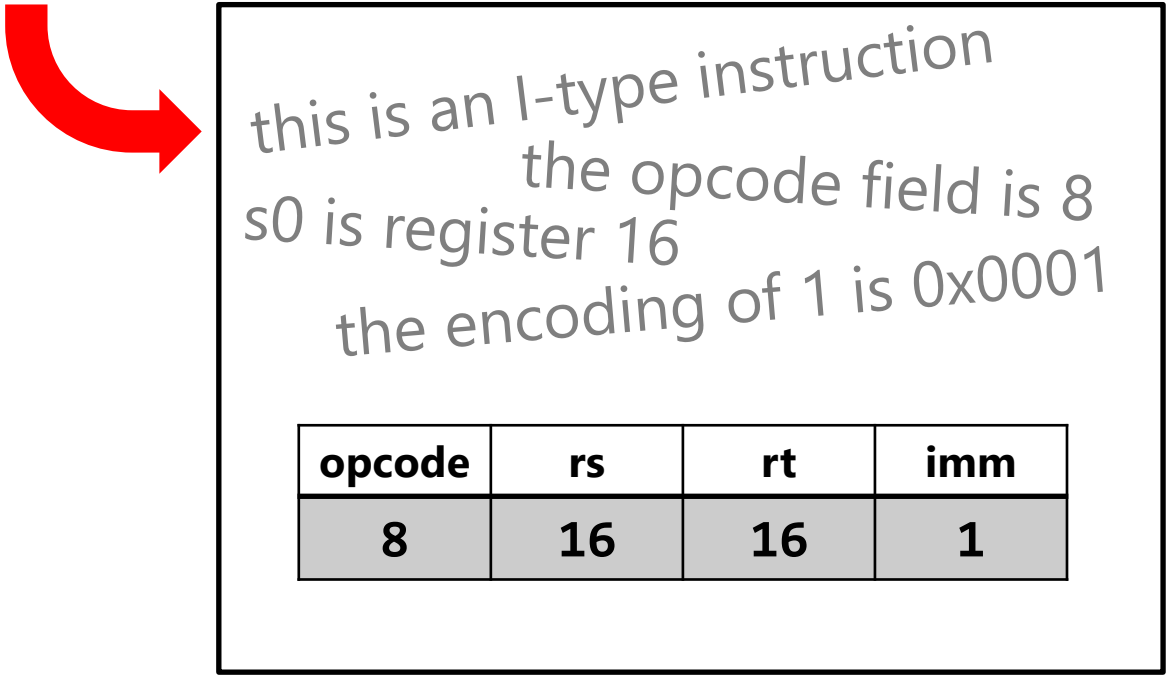
but, there's clearly a *little* more going on under the hood...

Address	Instruction
0x00400000	0x24100000
0x00400004	0x00102021
0x00400008	0x0C100008
0x0040000C	0x22100001
0x00400010	0x2A010005
0x00400014	0x1420FFFB
0x00400018	0x2402000A
0x0040001C	0x0000000C
0x00400020	0x24020001
0x00400024	0x0000000C
0x00400028	0x03E00008

How it works

- An assembler is a pretty simple program
- See an instruction, output its encoding

```
addi s0, s0, 1
```



opcode	rs	rt	imm
8	16	16	1

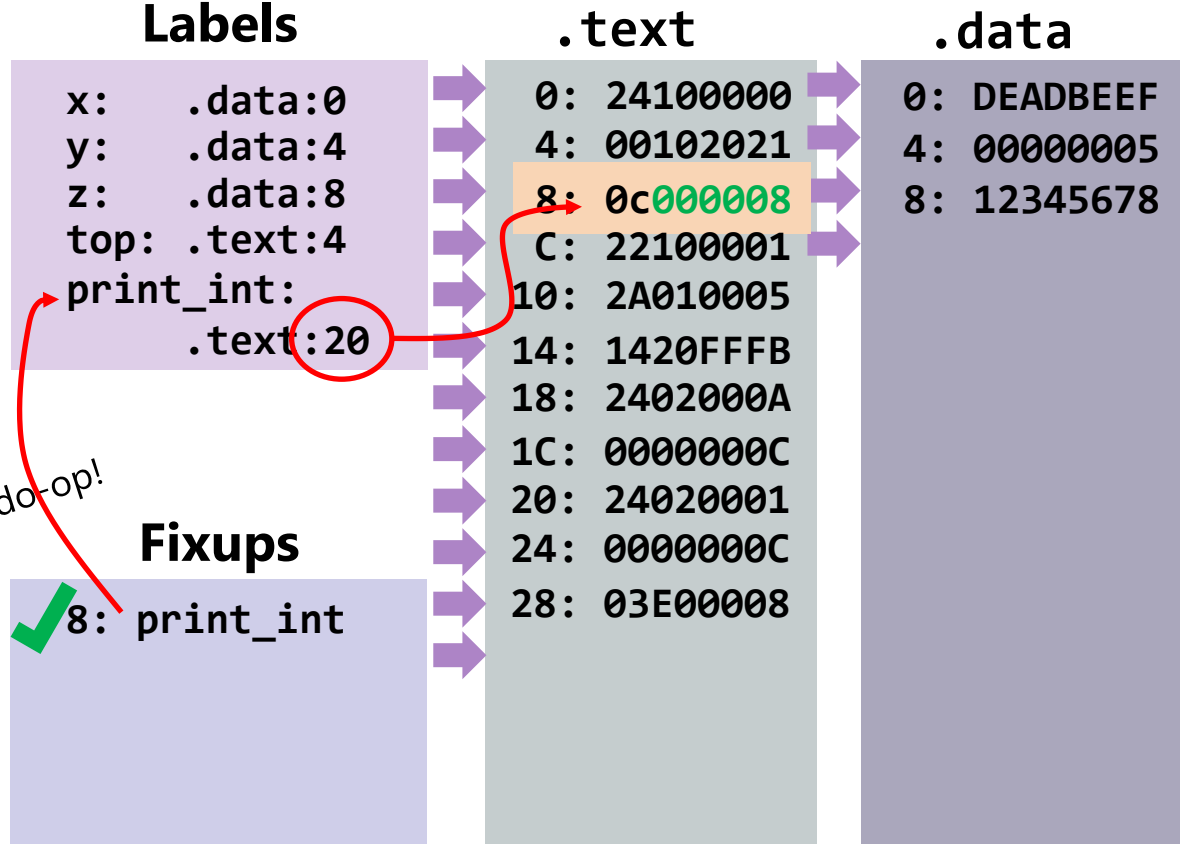
splut

```
0x22100001
```

But what about labels and the data segment

How it *actually* works (animated)

```
.data
x: .word 0xDEADBEEF
y: .word 5
z: .word 0x12345678
.text
    li   s0, 0
top:
    move a0, s0
    jal  print_int
    addi s0, s0, 1
    blt  s0, 5, top
    li   v0, 10
    syscall
print_int:
    li v0, 1
    syscall
    jr ra
```



then, run through the fixups!

Yum yum

- if a label doesn't exist, **it's an error.**
- now we have **machine code!**
- it's packaged up into a casing: an **object file**
- then the object files are **linked**
- and *then* you get an executable program
  - this is CS0449 stuff!



What about compilers?

- ahahaha oh they're a lot more complicated

```
int main(int argc, char** argv) {
    if(argc < 2)
        fatal("gimme arguments");
    else {
        ...
    }
}
```

lexing!

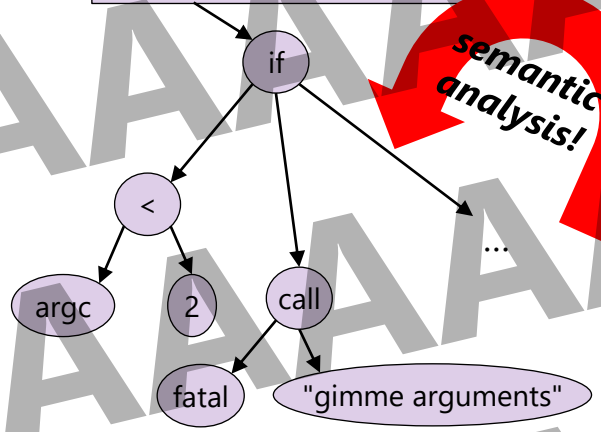
Tokens

```
KEYWORD("int"),
ID("main"),
LPAREN,
KEYWORD("int"),...
```

parsing!

AST (Abstract Syntax Tree)

```
Function
  ret_type: int
  name: "main"
  args: [
    {type: int, name: "argc"},
    {type: ptr(ptr(char)),
     name: "argv"}
  ]
```



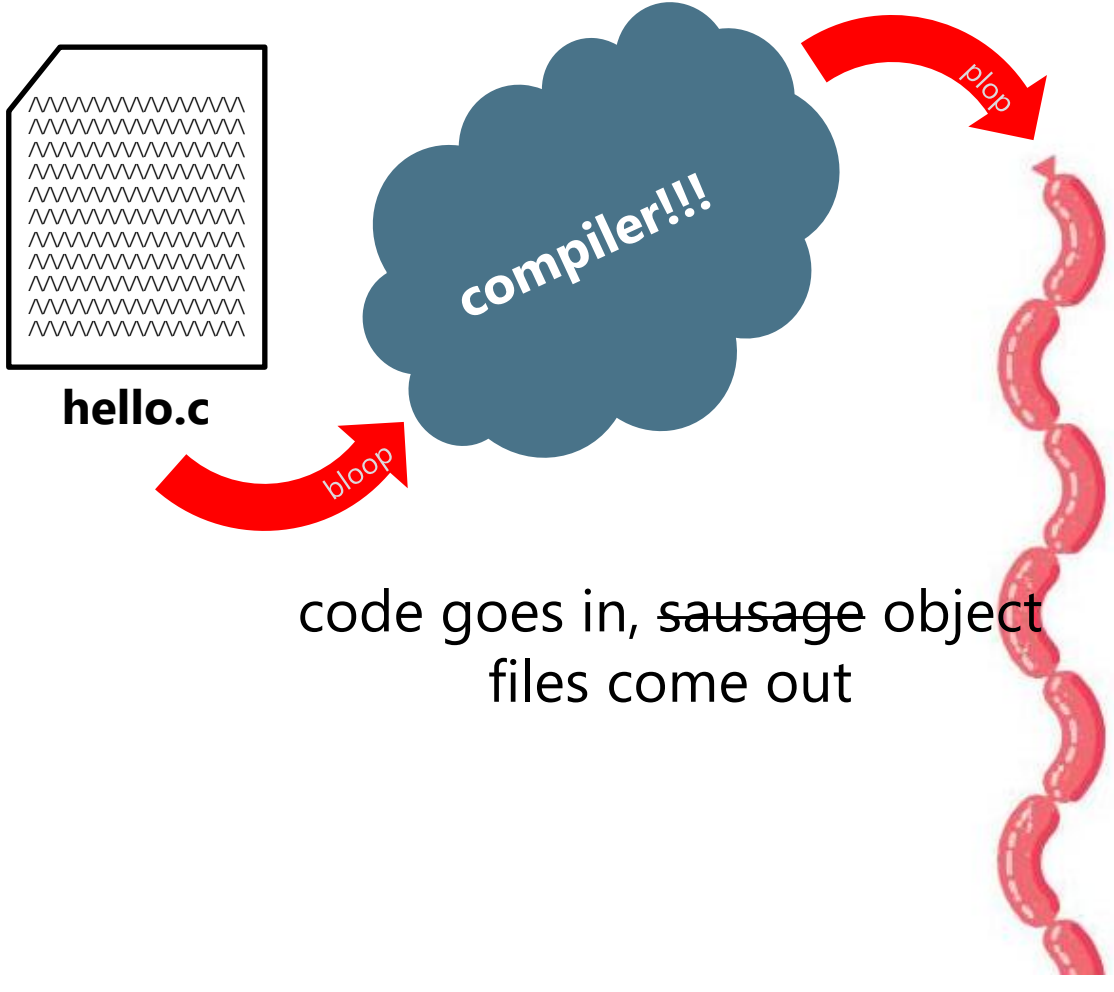
semantic analysis!

intermediate representation translation!

mysterious bullshit

It's just a grinder.

- all that really matters:



code goes in, ~~sausage~~ object files come out

some compilers output assembly and rely on an assembler to produce machine code

these days, it's common for the compiler itself to produce machine code, or some kind of platform-independent assembly code

Jumps and Branches

Maybe you never noticed…

- the control flow instructions are divided into two groups.

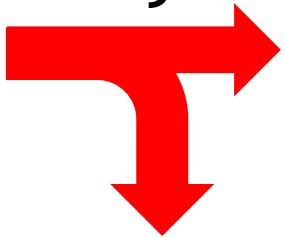
jumps make execution go to **one** specific place

```
j end
```



branches make execution go to **one of two** places

```
bne s1, t0, top
```



but.. why?

well, notice the **operands** of each.

A matter of practicality

- Each jump or branch has a **target:** where it goes to
- We'd like to be able to encode *any* target address…
- But we have a **fixed number of bits** to encode our instructions.

think about the cases where **jumps** are used.

```
some_function()
        return;

while(true) {}
```

how far away is a *jump* target likely to be?

now think about the cases where **branches** are used.

```
if(...) {}
        while(...) {}

for(...) {}
```

how far away is a *branch* target likely to be?

Absolute versus Relative

- we say that jumps are **absolute** and branches are **relative.**

```
top:
  move a0, s0
  jal  print_int
  add  s0, s0, 1
  blt  s0, 5, top
  ...
```

jumps just **set the PC to a new value.**

`PC = 0x800400B0`

`PC += (-16)`

branches either **add an offset to the PC** or do nothing.

jumps need a long address, but branches only need a small offset. so we can fit them into J and I instructions!

More bang for your buck

- **every MIPS instruction** is 4 bytes
- what's **memory alignment** again?

Address in hex	and in binary
0x78000000	0111 1000 0000 0000 0000 0000 0000 0000
0x78000004	0111 1000 0000 0000 0000 0000 0000 0100
0x78000008	0111 1000 0000 0000 0000 0000 0000 1000
0x7800000C	0111 1000 0000 0000 0000 0000 0000 1100

what do you notice about these low 2 bits?

in binary, **multiples of 4 always end in 00**

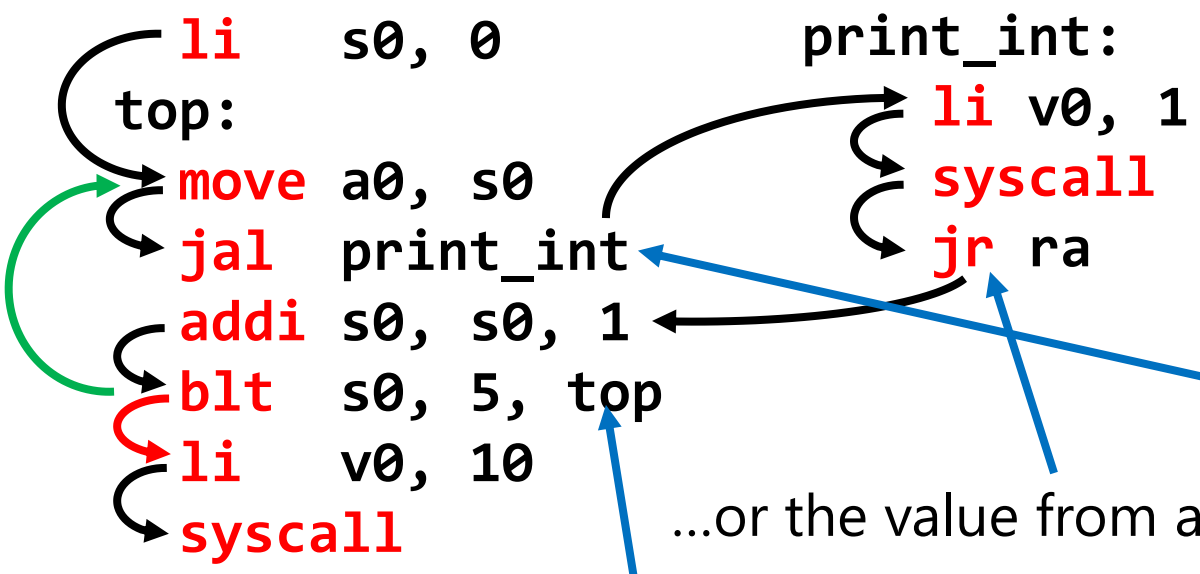
since every instruction's address ends in 00, do we need to *store* it?

Instruction Fetching

What do we do next, boss?

Remember this?

- what order do these instructions run?



most instructions change the PC to the **next address**

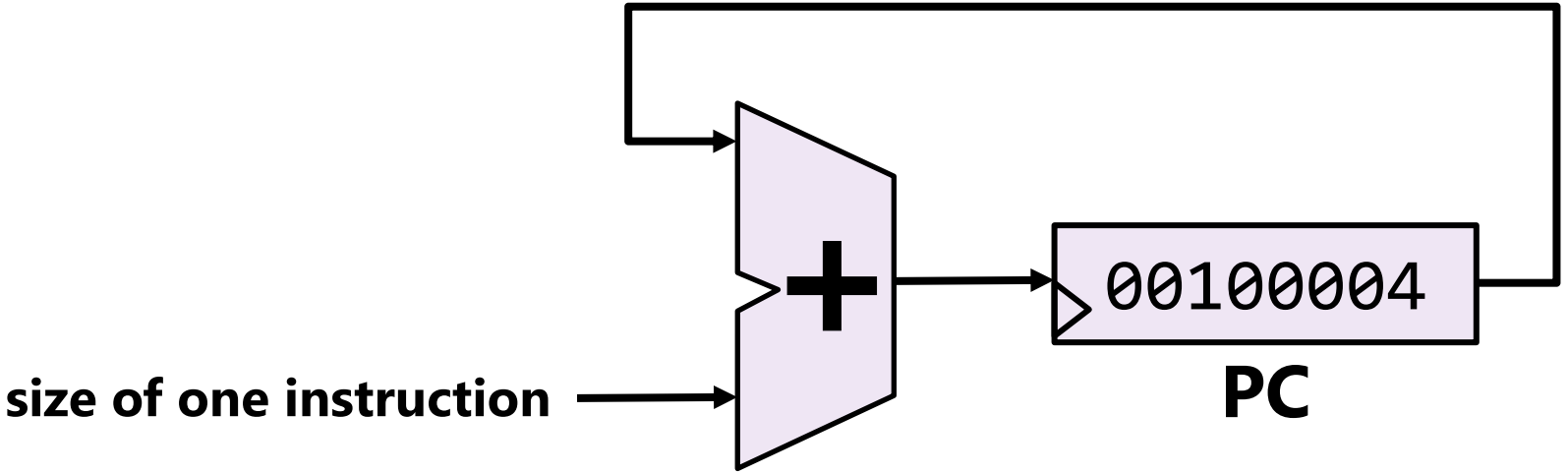
control flow instructions can change the PC to a constant...

...or the value from a register...

...or one of *two* choices, *conditionally*

Forwarrrrrrrd MARCH

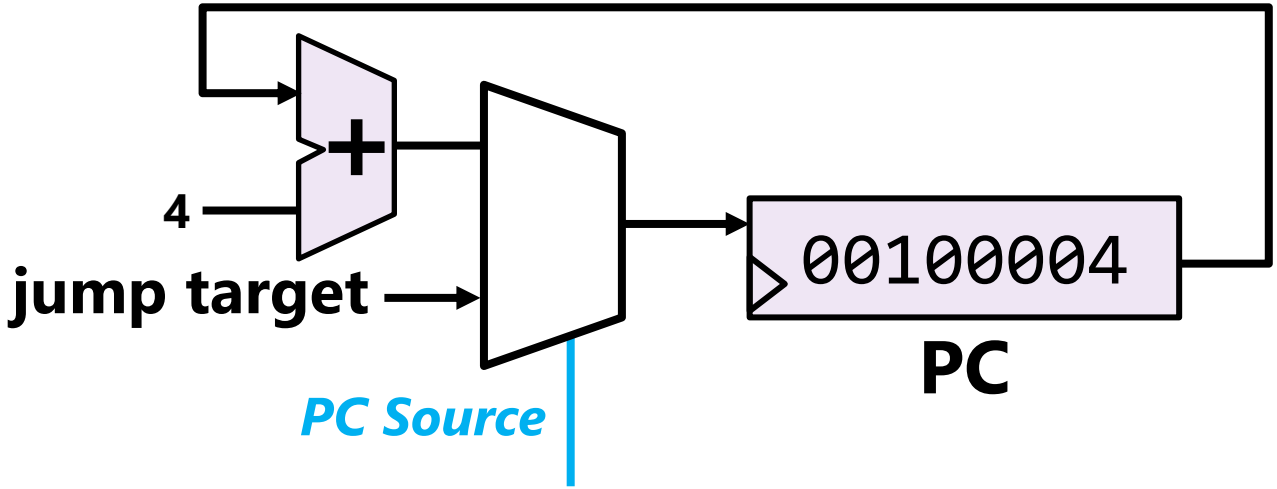
- moving ahead by 1 instruction each cycle is easy enough
  - This is a FSD, the next state is the current state + 1 :D



how big are instructions in MIPS?

Arbitrary locatiooooon MARCH

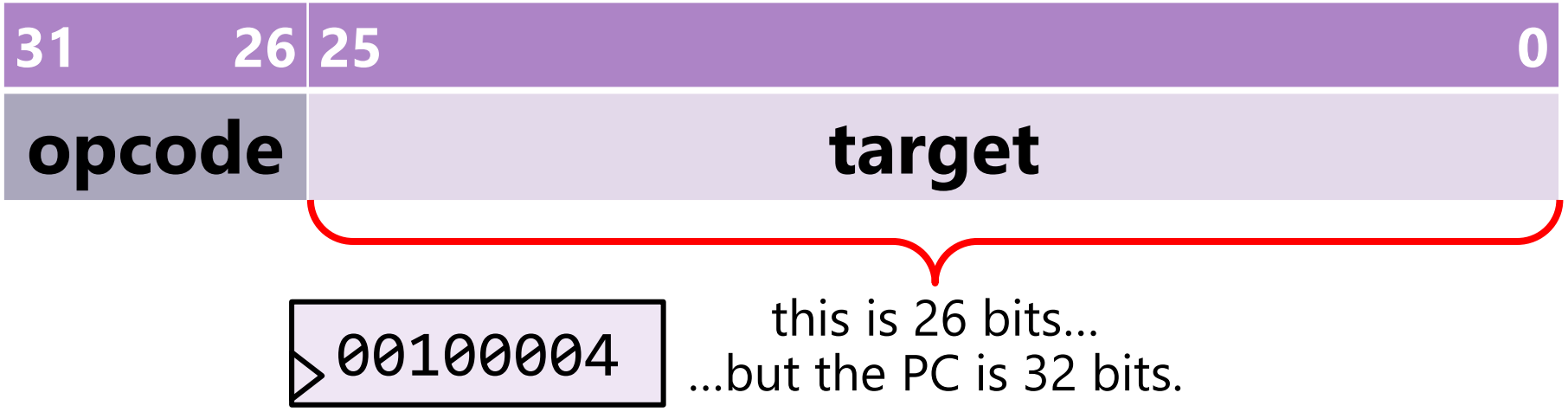
- **jumps** (j, jal, jr) put a **constant value** into the PC
  - we call this the **jump target.**
- well now we have *two* choices of where to go. how do we **choose?**



PC Source (***PCSrc*** for short) is a **control signal.**
many control signals are just MUX selectors

MIPS jump targets

- in MIPS, **j** and **jal** use the **J-type** instruction format:

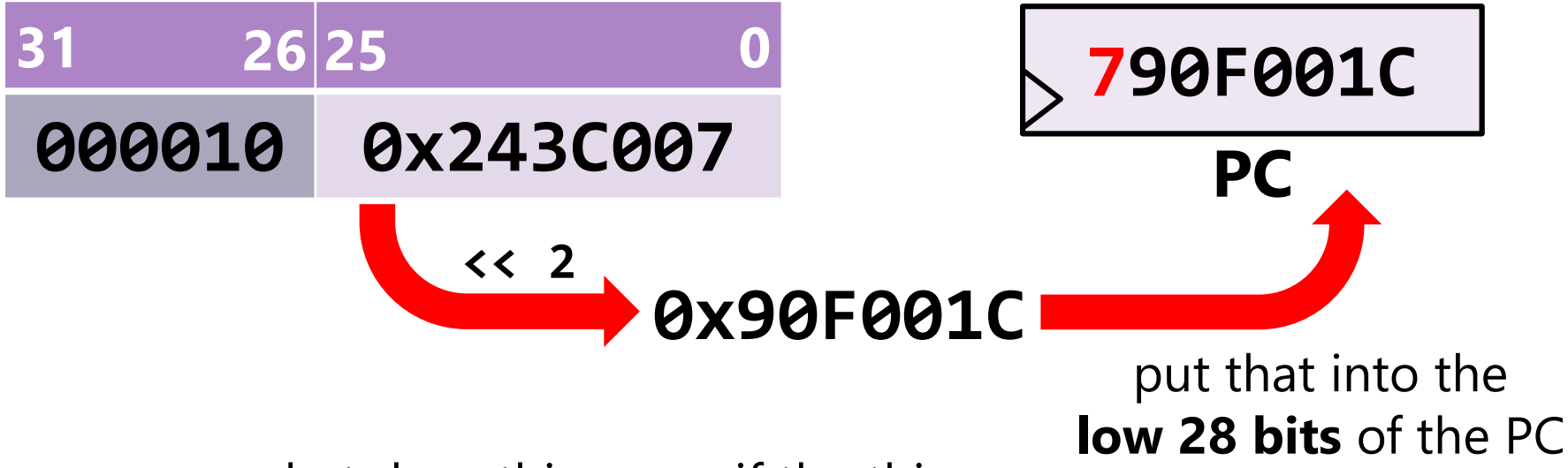


WHAT DO??

Do we *really* need a full 32-bit address (no)

- we don't need to store the **lower 2 bits** because of alignment.
- most programs are nowhere *near* big enough to need 32-bit addrs.
- so in MIPS, jumps only change the **lower 28 bits** of the PC.

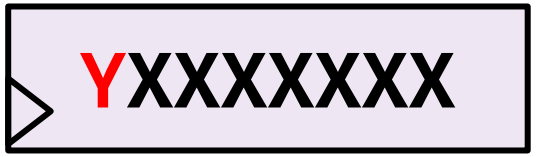
here's a **j.**



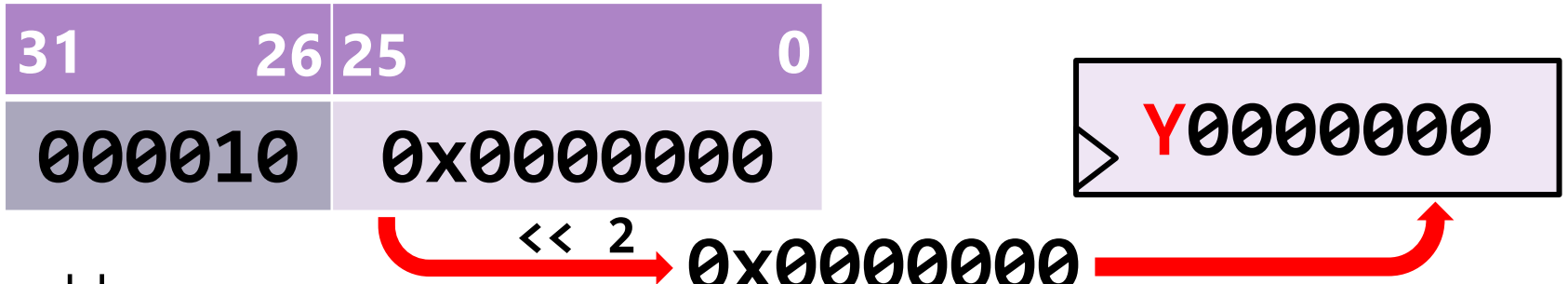
what does this mean if the thing you're jumping to is too far away?

Do we *really* need a full 32-bit address (no) (cntd.)

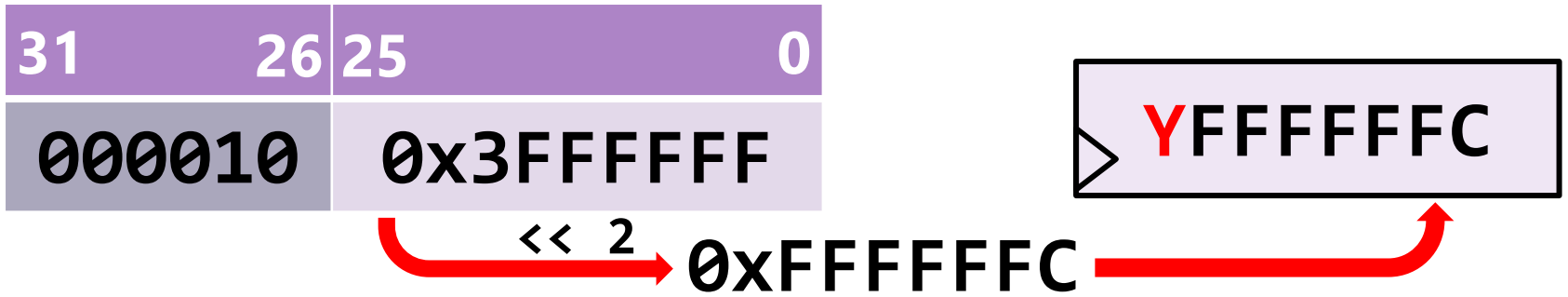
- If a jump instruction is in address



- It can reach from address:



- To address:



If jumping REALLY far, far away…

- What if we want to jump too far?

```
beq  t0, zero, a_label_far_far_away

j  a_label_far_far_away
```

- There is one instruction that can jump into a 32-bit address
  - What is that?
    - How big is register **ra**?

```
jr   ra
```

If jumping REALLY far, far away…

```
j  a_label_far_far_away
```



```
    la   t0, a_label_far_far_away
    jr   t0
```

```
beq  t0, zero, a_label_far_far_away
```



```
    bne  t0, zero, _skip_jump
    la   t0, a_label_far_far_away
    jr   t0
_skip_jump:
```

If jumping REALLY far, far away…

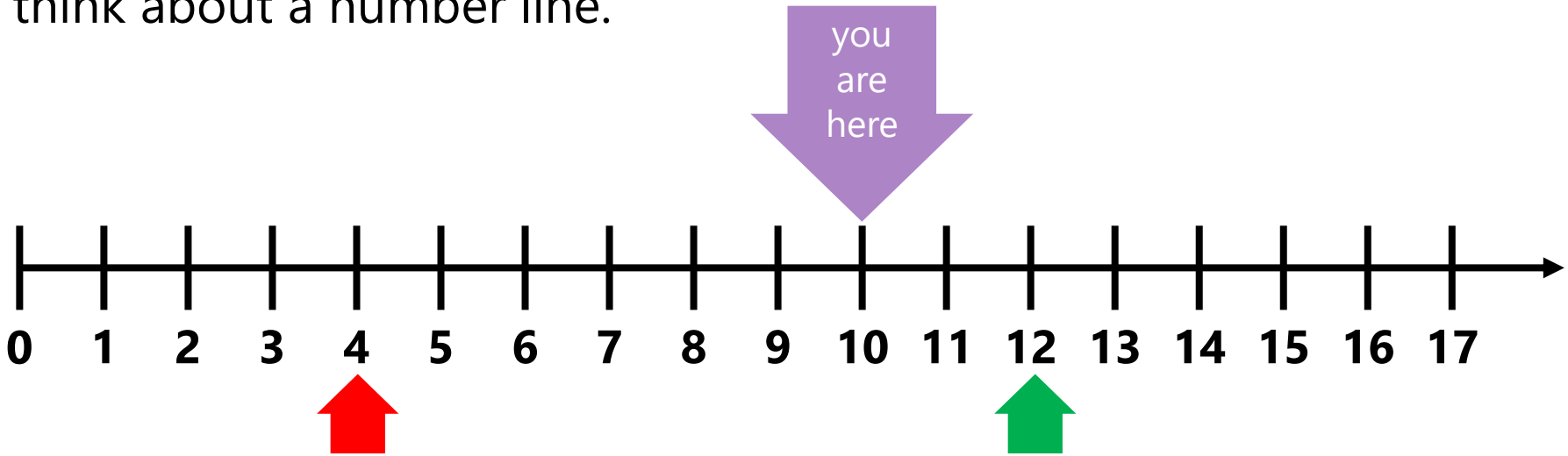
```
jal  a_label_far_far_away
```



```
la    t0, a_label_far_far_away
jalr  t0
```

Relative branches

- think about a number line.



if you want to get here, what do you have to **add to 10?**

how about here?

what's the pattern? **`destination - source`**

MIPS branch offsets

- In MIPS the PC points to the **next instruction to run.**
- let's say we're running the **beq** here. it's at address **00**...

...but the PC is here. PC

```
      00: beq a0, 10, else
      04: li  v0, 0
      08: b   end
else: 0C: li  v0, 1
end:  10: ...
```

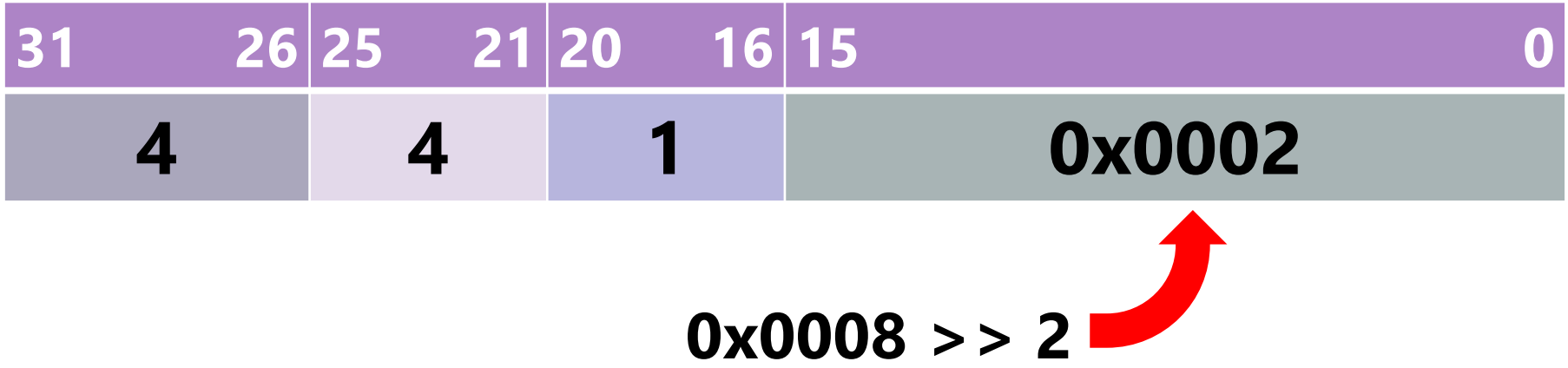
we want to get to address **0C**.
how do we get there?

PC += 8

the **branch offset** for this **beq** is:
target – (branch address + 4) = 12 – (0 + 4) = 8

Encoding it

- Since the branch's immediate is only 16 bits…



if the branch offset is **negative** like 0xFFFFFFE8, no big deal – chop off the top 16 bits. **0xFFE8 is still a negative number.**

The number stored is the number of instructions

- To go to else, from the updated value of PC: Jump 2 instructions down

the **branch offset** for this **beq** is:

target – (branch address + 4)

=

12 – (0 + 4) = 8

PC

```
      00: beq a0, 10, else
      04: li  v0, 0
      08: b   end
else: 0C: li  v0, 1
end:  10: ...
```

31 – 26	25 – 21	20 – 16	15 – 0
4	4	1	0x0002

0x0008 >> 2